

Assessment Report

Assessment Name: Olympic Database

Student Name : Ming Cheing Yee

Curtin Student ID : 22467577

Lab Group : 2 (Friday, 3pm-5pm)

Introduction

This report aims to provide an explanation of the developing and implementing of the database called Olympic_22467577. The selected scenario of this database is the Olympic Games 2024, which took place from 26 July to 11 August 2024, in Paris, France. The users can store and retrieve data, including athletes, officials, events, venues, countries, medals, participation, and event management, from the database for any academic purpose. As a result, it will serve as a valuable resource for any researchers, students, or even future Olympic Games organizers to make analysis and interpretations on data that is related to the Olympics.

During the development of the database, I have done many activities, including identifying entities, constraints, and relationships in the database's structure, designing an entity-relationship (ER) diagram and also relational schema to ensure their normality, and lastly, searching for valid resources about the Olympic Games 2024 data. Furthermore, I use MySQL to implement the database on Linux and insert pieces of valid data into the database.

This report will show all the details of the designing process of the database, including the reasons for the selection of entities and attribute's data types, the implementation process, various queries, and the advanced features developed, which are the stored procedures and triggers that were used to increase the efficiency of the uses of the database. The aim of this report is to provide a clear insight into the database's functionality and its significance in supporting Olympic-related data management.

Database Designing

I. Entities selected

Entity Sets	Primary Keys	Other Attribute	Reason of selection
Athletes	Athlete_Code	Name, Gender, Birth_Date, Discipline, Side_Job	The main participant of the Olympic
Countries	Country_Code	Name	The land that athletes belong to
Events	Sport_Code	Name	Each competition has its own event
Venues	Venue_Code	Name	The location where events take place during competition
Officials	Official_Code, Belonged Organization	Name, Gender, Role	People play a role to oversee the events
Medals (Weak Entity)	Medal_Code (Weak Key)	Medal_Type	The result of the events

II. Relationship selected

Relationship Sets	Related Entity Sets	Other Attribute	Reason of selection
Participate	Athletes & Events	Started_Date, Ended_Date	Athletes participate events
Held in	Events & Venues		Events held in venues
Managed by	Events & Officials		Events managed by officials
Belongs to	Athletes & Countries		Athletes belonged to country
Wins (Weak Relationship)	Athletes & Medals	Awarded_Date	Athletes win medal

III. Relationship constraints

Relationship Sets	Cardinality Constraints	Participation Constraints
Participate	Many to Many → One athlete can participate many events → One event can be participated by many athlete	Athlete ->Total (Athlete cannot exist without event) Event -> Partial (Event can exist without athlete)
Held in	One to Many → One event can hold in only one venue → One venue can hold many events	Event ->Partial (Event can exist without venue) Venue->Partial (Venue can exist without event)
Managed by	Many to Many → One event can be managed by many officials → One official can manage many event	Event->Partial (Event can exist without official) Official->Total (Official cannot exist without event)
Belongs to	One to Many → One athlete belongs to one country only → One country can have many athletes	Athlete->Total (Athlete cannot exist without country) Country->Partial (Country can exist without athlete)
Wins (Weak Relationship)	One to Many → One athlete can award many medals → One medal can only be awarded by one athlete	Athlete->Total (Athlete cannot exist without medal) Medal->Partial (Medal can exist without athlete)

IV. ER Model

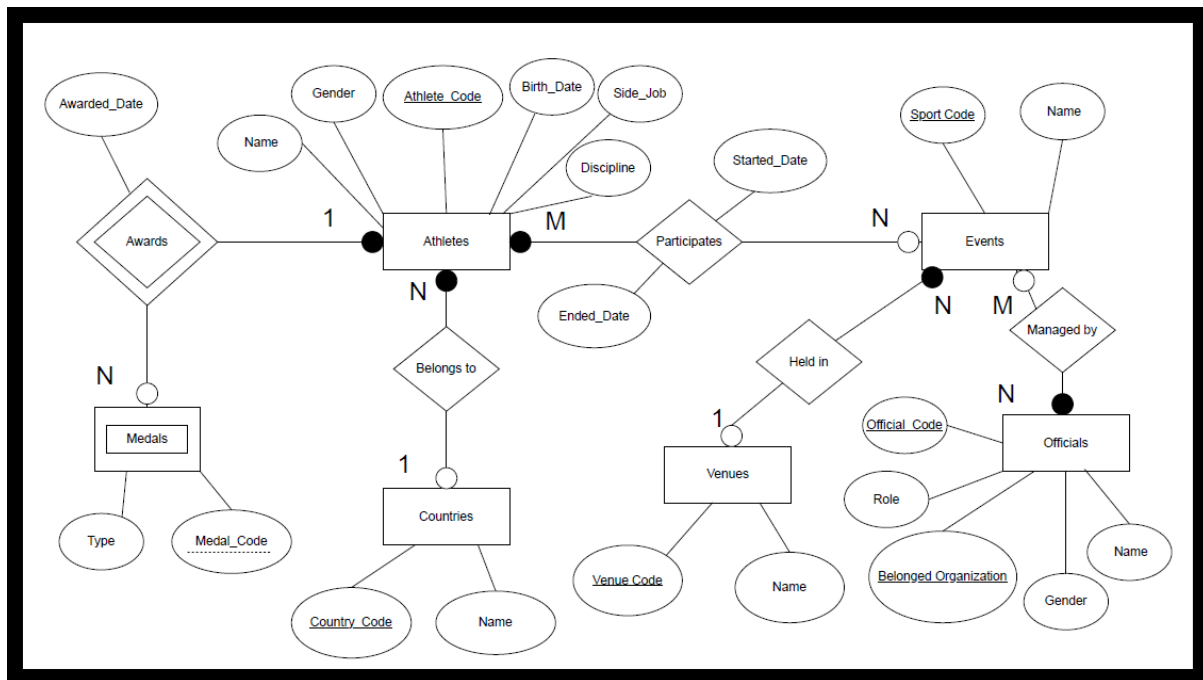


Figure 1- ER MODEL

This ER model is produced using Chen notation, which visualizes the database structure as a reference during the implementation and helps future designers make changes or updates to my database if needed

V. Relational schema

Venue (Venue_Code, Name)

Country (Country_Code, Name)

Athlete (Athlete_Code, Name, Gender, Discipline, Birth_Date, Side_Job, Country_Code)

FK Country_Code REF Country (Country_Code)

Event (Sport_Code, Name, Venue_Code)

FK Venue_Code REF Venue (Venue_Code)

Medal (Athlete_Code, Medal_Code, Type)

FK Athlete_Code REF Athlete (Athlete_Code)

Participates (Athlete_Code, Sport_Code, Started_Date, Ended_Date)

FK Athlete_Code REF Athlete (Athlete_Code)

FK Sport_Code REF Event (Sport_Code)

Official (Official_Code, Belonged Organization, Name, Gender, Role)

Managed by (Official_Code, Belonged Organization, Sport_Code)

FK Official_Code REF Official (Official_Code)

FK Belonged_Organization REF Official (Belonged_Organization)

FK Sport_Code REF Event (Sport_Code)

Normality Check

i. In First Normal Form

➔ No multivalued attribute in every entity

ii. In Second Normal Form

➔ Already in first normal form

➔ Partial dependency eliminated

➔ Non-key attribute only depends on primary key

iii. In Third Normal Form

➔ Already in second normal form

➔ No non-key attribute depends on another non-key attribute

VI. Data Description

Venue (Venue Table)

COLUMN NAME	DATA TYPE	SIZE	Constraints	DESCRIPTION
Venue_Code	CHAR	3	PRIMARY KEY	Identify Code for Each Venue used
Name	VARCHAR	50	NOT NULL	Name of Each Venue used

Country (Country Table)

COLUMN NAME	DATA TYPE	SIZE	Constraints	DESCRIPTION
Country_Code	CHAR	3	PRIMARY KEY	Identify Code for Each Country Joined
Name	VARCHAR	40	NOT NULL	Name of Each Country Joined

Athlete (Athlete Table)

COLUMN NAME	DATA TYPE	SIZE	Constraints	DESCRIPTION
Athlete_Code	INT	7	PRIMARY KEY	Identify Code for each athlete participated
Name	VARCHAR	50	NOT NULL	Name of each athlete
Gender	VARCHAR	6	NOT NULL	Gender for each athlete (Male/Female)
Discipline	VARCHAR	50		The sport an athlete focusses on and compete
Birth_Date	DATE		CHECK (Birth_Date >= '1900-01-01')	Date of Born (Athlete)
Side_Job	VARCHAR	50		The other job an athlete does besides the athlete
Country_Code	CHAR	3	NO NULL FOREIGN KEY	Identify the nationality of Athlete

Medal (Medal Table)

COLUMN NAME	DATA TYPE	SIZE	Constraints	DESCRIPTION
Athlete_Code	INT	7	FOREIGN KEY PRIMARY KEY	Identify Code of Athlete get the medal
Medal_Code	INT	1	PRIMARY KEY	Identify Code of Medal (1 for Gold,2 for Silver,3 for Bronze)
Medal_Type	CHAR	6	NOT NULL	The material used to make medal
Award_Date	DATE			The date an athlete gets the medal

Event (Event Table)

COLUMN NAME	DATA TYPE	SIZE	Constraints	DESCRIPTION
Sport_Code	CHAR	3	PRIMARY KEY	Identify Code of sport type
Name	VARCHAR	50	NOT NULL	Name of sport type competed
Venue_Code	CHAR	3	FOREIGN KEY NOT NULL	Identify Code of Venue used to hold an event (All the event of same sport type holds in the same venue)

Participation (Participates Relationship Table)

COLUMN NAME	DATA TYPE	SIZE	Constraints	DESCRIPTION
Athlete_Code	INT	7	FOREIGN KEY NOT NULL	Identify Code of Athlete participate in an event
Sport_Code	CHAR	3	FOREIGN KEY NOT NULL	Identify Code of Event an athlete has participated
Started_Date	DATE			The date of an event started
Ended_Date	DATE			The date of an event ended

Official (Officials Table)

COLUMN NAME	DATA TYPE	SIZE	Constraints	DESCRIPTION
Official_Code	INT	7	PRIMARY KEY	Identify Code of Official
Name	VARCHAR	50	NOT NULL	Name of each Official
Gender	CHAR	6		Gender of each Official (Male/Female)
Role	VARCHAR	30		Function of official in an event
Belonged_Organization	VARCHAR	80	PRIMARY KEY	Name of Organization of an Official belonged

Event_Management (Manages relationship Table)

COLUMN NAME	DATA TYPE	SIZE	Constraints	DESCRIPTION
Official_Code	int	7	FOREIGN KEY NOT NULL	Identify Code of Official that manage an event
Belonged_Organization	VARCHAR (80)	50	FOREIGN KEY NOT NULL	Name of Organization of an Official (manage an event) belonged
Sport_Code	CHAR	3	FOREIGN KEY NOT NULL	Identify Code of Sport of event that an official manages

Implementation

In order to implement the database, I create four SQL script:

1) SetUpDatabase.sql

➔ Consists the commands that create Olympic_22467577 database



```
1 #YeeMingCheing_22467577
2
3 #Name:Yee Ming Cheing
4 #StudentID:22467577
5 #Database System 2024 Sem2
6 #Purpose:Setting up the database
7
8 #Create Database called Olympic_22467577
9 DROP DATABASE IF EXISTS Olympic_22467577;
10 CREATE DATABASE Olympic_22467577;
11
12 #Use the Database created just now
13 USE Olympic_22467577;
14
```

Figure 2-Section of command Details of SetUpDatabase.sql

2) SetUpDataTable.sql

➔ Consists the commands that create all the related table into the Olympic_22467577 database



```
1 #YeeMingCheing_22467577
2
3 #Name:Yee Ming Cheing
4 #StudentID:22467577
5 #Database System 2024 Sem2
6 #Purpose:Setting up the datatable
7
8 #Create the Country Table
9 DROP Table IF EXISTS Country;
10 CREATE table Country(
11     Country_Code CHAR(3) PRIMARY KEY, -- Set Country Code As Primary Key
12     Name VARCHAR(40) NOT NULL
13 );
14
15 #Create the Venue Table
16 DROP Table IF EXISTS Venue;
17 CREATE table Venue(
18     Venue_Code CHAR(3) PRIMARY KEY, -- Set Venue Code As Primary Key
19     Name VARCHAR(50) NOT NULL
20 );
21
22 #Create the Athlete Table
23 DROP Table IF EXISTS Athlete;
24 CREATE table Athlete(
25     Athlete_Code Int(7) PRIMARY KEY, -- Set Athlete Code As Primary Key
26     Name VARCHAR(50) NOT NULL,
27     Gender VARCHAR(6) NOT NULL,
28     Discipline VARCHAR(50),
29     Birth_Date DATE CHECK (Birth_Date >= '1900-01-01'), -- Use CHECK Constraint to ensure a valid athlete's age
30     Side_Job VARCHAR(50) DEFAULT 'None', -- Set the Default Value of Side_Job is 'None' for the athlete who does not have side job
31     Country_Code CHAR(3) NOT NULL,
32     CONSTRAINT fk_country
33     FOREIGN KEY (Country_Code)
34     REFERENCES Country(Country_Code) -- Country Code from Country table become foreign key
35     ON DELETE CASCADE ON UPDATE CASCADE -- Delete or Update on Country Code in Country Table will change the Country Code in Athlete Table
36 );
37
38 #Create the Medal Table
```

Figure 3-Section of command Details of SetUpDataTable.sql

3) SetUpData.sql

- ➔ Consists the commands that insert data into all the table in the Olympic_22467577 database
- ➔ Data source: <https://www.kaggle.com/datasets/piterfm/paris-2024-olympic-summer-games>
- ➔ I) Country table -> 224 rows of data
- II) Venue table -> 34 rows of data
- III) Athlete table -> 400 rows of data
- IV) Medal table -> 10 rows of data
- V) Event table -> 34 rows of data
- VI) Participation table -> 400 rows of data
- VII) Official table -> 545 rows of data
- VIII) Event_Management table -> 545 rows of data

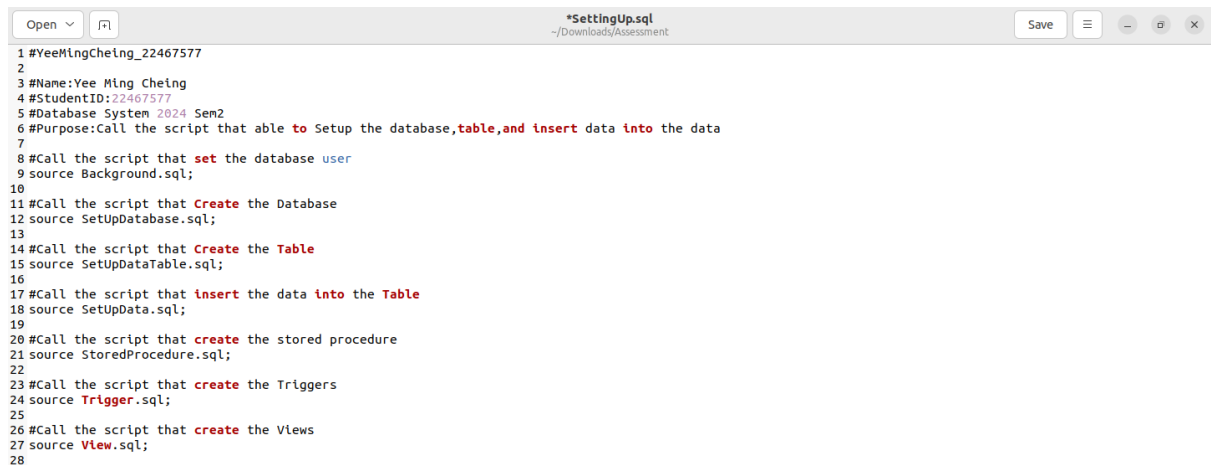


```
1 #YeeMingCheIng_22467577
2
3 #Name:Yee Ming CheIng
4 #StudentID:22467577
5 #Database System 2024 Sem2
6 #Purpose:Insert Data into all the table created
7
8 #Insert Data into Country Table
9 INSERT INTO Country VALUES ('AFG','Afghanistan'),
10 ('AHO','Netherlands Antilles'),
11 ('AIN','AIN'),
12 ('ALB','Albania'),
13 ('ALG','Algeria'),
14 ('AND','Andorra'),
15 ('ANG','Angola'),
16 ('ANT','Antigua and Barbuda'),
17 ('ARG','Argentina'),
18 ('ARM','Armenia'),
19 ('ARU','Aruba'),
20 ('ASA','American Samoa'),
21 ('AUS','Australia'),
22 ('AUT','Austria'),
23 ('AZE','Azerbaijan'),
24 ('BAH','Bahamas'),
25 ('BAN','Bangladesh'),
26 ('BAR','Barbados'),
27 ('BDI','Burundi'),
28 ('BEL','Belgium'),
29 ('BEN','Benin'),
30 ('BER','Bermuda'),
31 ('BHU','Bhutan'),
32 ('BIH','Bosnia and Herzegovina'),
33 ('BIZ','Belize'),
34 ('BLR','Belarus'),
35 ('BOC','BOC'),
36 ('BOL','Bolivia'),
37 ('BOT','Botswana'),
38 ('BRA','Brazil'),
```

Figure 4-Section of command Details of SetUpData.sql

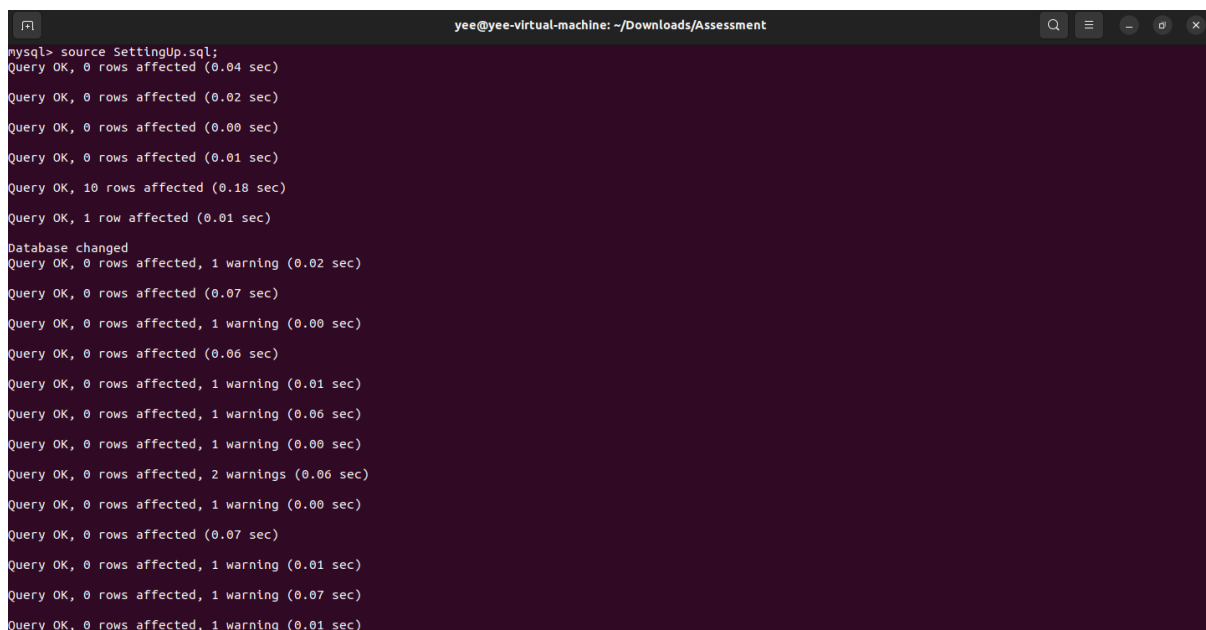
4) SettingUp.sql

- ➔ Consists of the commands that call all the SQL scripts related to database implementation, such as SetUpDatabase.sql or SetUpDataTable.sql
- ➔ The user only needs to call this SQL script to complete the implementation on the Olympic_22467577 database



```
1 #YeeMingCheing_22467577
2
3 #Name:Yee Ming Cheing
4 #StudentID:22467577
5 #Database System 2024 Sem2
6 #Purpose:Call the script that able to Setup the database,table,and insert data into the data
7
8 #Call the script that set the database user
9 source Background.sql;
10
11 #Call the script that Create the Database
12 source SetUpDatabase.sql;
13
14 #Call the script that Create the Table
15 source SetUpDataTable.sql;
16
17 #Call the script that insert the data into the Table
18 source SetUpData.sql;
19
20 #Call the script that create the stored procedure
21 source StoredProcedure.sql;
22
23 #Call the script that create the Triggers
24 source Trigger.sql;
25
26 #Call the script that create the Views
27 source View.sql;
28
```

Figure 5-Command Details of SettingUp.sql



```
mysql> source SettingUp.sql;
Query OK, 0 rows affected (0.04 sec)

Query OK, 0 rows affected (0.02 sec)

Query OK, 0 rows affected (0.00 sec)

Query OK, 0 rows affected (0.01 sec)

Query OK, 10 rows affected (0.18 sec)

Query OK, 1 row affected (0.01 sec)

Database changed
Query OK, 0 rows affected, 1 warning (0.02 sec)

Query OK, 0 rows affected (0.07 sec)

Query OK, 0 rows affected, 1 warning (0.00 sec)

Query OK, 0 rows affected (0.06 sec)

Query OK, 0 rows affected, 1 warning (0.01 sec)

Query OK, 0 rows affected, 1 warning (0.06 sec)

Query OK, 0 rows affected, 1 warning (0.00 sec)

Query OK, 0 rows affected, 2 warnings (0.00 sec)

Query OK, 0 rows affected, 1 warning (0.00 sec)

Query OK, 0 rows affected (0.07 sec)

Query OK, 0 rows affected, 1 warning (0.01 sec)

Query OK, 0 rows affected, 1 warning (0.07 sec)

Query OK, 0 rows affected, 1 warning (0.01 sec)
```

Figure 6-Evidence of Implementation using SettingUp.sql

Use of Database

I. Querying

- ➔ 8 questions are prepared for the query implementation
- ➔ All the queries are stored in the Querying.sql script

1. Which athlete has a side job and is aged between 19 and 29?
 - ➔ Shows the percentage of athlete that able to balance sport and career after graduated from high school

```
SELECT Athlete_Code, Name, Side_Job  
  
FROM Athlete  
  
WHERE Side_Job IS NOT NULL AND  
  
Birth_Date BETWEEN '1995-01-01' AND '2005-12-31'
```

Figure 7-Query for Question1

```
mysql> SELECT Athlete_Code, Name, Side_Job  
-> FROM Athlete  
-> WHERE Side_Job IS NOT NULL AND  
-> Birth_Date BETWEEN '1995-01-01' AND '2005-12-31'  
-> ;
```

Athlete_Code	Name	Side_Job
1533136	BASS BITTAYE Gina Marian	police officer (sub-inspector)
1533176	CAMARA Ebrahima	prison officer
1533235	ARELLANO Fernanda	student
1533237	TOSCANO Pamela	sport soldier
1533364	ISMAIL Mohamed	sport soldier
1533371	HASSAN Ibrahim	soldier with the Djiboutian Republican Guard
1533372	WAISS Abdi	sport soldier
1533374	HASSAN NOUR Samiyah	sport soldier
1535191	GONZALEZ Alegna	serves in the navy
1535226	AGUILAR CHAVEZ PEON Mariana	student
1535310	BLANCO Luisa	student
1535343	WILLARS VALDEZ Randal	Armed forces
1535346	OROZCO LOZA Alejandra	Armed forces
1535348	AGUNDEZ GARCIA Gabriela	Armed forces
1535364	CARRILLO Duilio	Armed Forces
1535367	SOUZA Daniela Paola	Military
1535385	HOUSSEIN Alexandre	student
1535395	AL SAYEGH Safia	student
1535430	GRANDE Matias	student
1535438	RAMIREZ RAMOS Edson Ismael	student
1535441	QUEZADA MARQUEZ Carlos	student
1535505	LINARES Natalia	student

Sample Answer for Question 1

2. Which official comes from a sports federation?

➔ Shows the percentage of official from a sport federation

```
SELECT Official_Code, Name, Role
FROM Official
WHERE Belonged_Organization LIKE '%Federation%';
```

Figure 8-Query for question2

```
mysql> SELECT Official_Code, Name, Role
-> FROM Official
-> WHERE Belonged_Organization LIKE '%Federation%';
```

Official_Code	Name	Role
1550642	FUMEA Maria	Judge
1550643	LEUNG Li Li	Judge
1550645	MIAO Zhongyi	Judge
1550646	VALENZO AOKI Naomi Chieko	Judge
1550647	AL HITMI Ali	Judge
1550648	AL TABBAA Mouhammed Youssef	Judge
1550649	CELEN Suat	Judge
1550650	ESAWY Ehab	Judge
1550651	YAGI KITAGAWA Tammy	Judge
1550652	VALERIO Rogerio	Judge
1550653	MOWBRAY Denis	Judge
1550654	PEGAN Aljaz	Judge
1550655	PENICHE FRANCO Alejandro	Judge
1550656	PERRIERE Charles	Judge
1550657	TAEYMANS Rosy	Judge
1571122	ALSHATHRI A Rahman	Judge
1571123	KIM Nellia	Judge
1571124	TITOV Vasilii	Judge
1893012	CHIARI Roberto	Referee
1893016	THOMSON Mike	Referee
1893040	DUPUCHE Crystal	Referee
1893057	VACHERA Alejandro	Referee

Sample Answer for Question 2

3. How many athletes in the Olympics does a country have?

➔ The data obtained can be used in analysis the overall performance of a country

```
SELECT Country.Name, Country.Country_Code, COUNT(Athlete.Country_Code)
AS 'Number of athlete represented'
FROM Country INNER JOIN Athlete ON Country.Country_Code = Athlete.Country_Code
Group BY Country.Country_Code , Country.Name
ORDER BY COUNT(Athlete.Country_Code) DESC;
```

Figure 9-Query for Question 3

```
mysql> SELECT Country.Name, Country.Country_Code, COUNT(Athlete.Country_Code)
-> AS 'Number of athlete represented'
-> FROM Country INNER JOIN Athlete ON Country.Country_Code = Athlete.Country_Code
-> Group BY Country.Country_Code , Country.Name
-> ORDER BY COUNT(Athlete.Country_Code) DESC;
```

Name	Country_Code	Number of athlete represented
Mexico	MEX	83
Argentina	ARG	58
India	IND	42
Ireland	IRL	26
Islamic Republic of Iran	IRI	25
Ethiopia	ETH	20
Iraq	IRQ	16
Singapore	SGP	13
Romania	ROU	13
Colombia	COL	10
Puerto Rico	PUR	9
United Arab Emirates	UAE	7
Djibouti	DJI	7
Jamaica	JAM	6
Armenia	ARM	5
Guyana	GUY	5
Saint Lucia	LCA	4
Malaysia	MAS	4
Côte d'Ivoire	CIV	4
AIN	AIN	4
Comoros	COM	3
Congo	CGO	3
Jordan	JOR	3
Marshall Islands	MHL	3
Uzbekistan	UZB	3
Oman	OMA	2
Saint Kitts and Nevis	SKN	2
Malawi	MAW	2
St Vincent and the Grenadines	VIN	2
Libya	LBA	2

Sample Answer for Question 3

4. How many competitions have been held in a venue?

➔ The data obtained can be used for resource allocation in future Olympic events

```
SELECT Venue.Venue_Code, Venue.Name, COUNT(Event.Venue_Code) AS 'Total number of uses'
FROM Venue LEFT OUTER Join Event ON Venue.Venue_Code = Event.Venue_Code
GROUP BY Venue.Venue_Code, Venue.Name
ORDER BY COUNT(Event.Venue_Code) DESC;
```

Figure 10-Query for Question 4

```
mysql> SELECT Venue.Venue_Code, Venue.Name, COUNT(Event.Venue_Code) AS 'Total number of uses'
-> FROM Venue LEFT OUTER Join Event ON Venue.Venue_Code = Event.Venue_Code
-> GROUP BY Venue.Venue_Code, Venue.Name
-> ORDER BY COUNT(Event.Venue_Code) DESC;
```

Venue_Code	Name	Total number of uses
SP	South Paris Arena	4
VN	Vaires-sur-Marne Nautical Stadium	3
BCY	Bercy Arena	2
CDM	Champ de Mars Arena	2
GRP	Grand Palais	2
STA	Stade de France	2
LC	La Concorde	2
RGA	Stade Roland-Garros	2
MAM	Marseille Marina	1
YDM	Yves-du-Manoir Stadium	1
VER	Château de Versailles	1
VE2	Saint-Quentin-en-Yvelines BMX Stadium	1
VE1	Saint-Quentin-en-Yvelines Velodrome	1
TRO	Trocadéro	1
TAH	Teahupo'o, Tahiti	1
NPA	North Paris Arena	1
AQC	Aquatics Centre	1
BOR	Bordeaux Stadium	1
CPL	Porte de La Chapelle Arena	1
CTX	Chateauroux Shooting Centre	1
DEF	Paris La Defense Arena	1
EIF	Eiffel Tower Stadium	1
LBO	Le Bourget Sport Climbing Venue	1
INV	Invalides	0
ELA	Elancourt Hill	0
STE	Geoffroy-Guichard Stadium	0
LYO	Lyon Stadium	0
LGN	Le Golf National	0
PDP	Parc des Princes	0
LIL	Pierre Mauroy Stadium	0
NIC	Nice Stadium	0
NAN	La Beaujoire Stadium	0

Sample Answer for Question 4

5. Which athletes were born with the same data?

➔ Shows some interesting coincidences which can be used in promotion or news

```
SELECT DISTINCT A1.Athlete_Code,A1.Name,A1.Gender,A1.Birth_Date
FROM Athlete AS A1 JOIN Athlete AS A2 ON A1.Birth_Date = A2.Birth_Date AND
A1.Athlete_Code <> A2.Athlete_Code
ORDER BY A1.Birth_Date;
```

Figure 11-Query for Question 5

```
mysql> SELECT DISTINCT A1.Athlete_Code,A1.Name,A1.Gender,A1.Birth_Date
-> FROM Athlete AS A1 JOIN Athlete AS A2 ON A1.Birth_Date = A2.Birth_Date AND
-> A1.Athlete_Code <> A2.Athlete_Code
-> ORDER BY A1.Birth_Date;
```

Athlete_Code	Name	Gender	Birth_Date
1535511	RUIZ HURTADO Flor Denis	Female	1991-01-29
1537168	CASTRO RIVERA Luis	Male	1991-01-29
1537012	SAIKHOM Mirabai Chanu	Female	1994-08-08
1535356	MORENO Alexa	Female	1994-08-08
1538146	CHAUHAN Maheshwari	Female	1996-07-04
1539952	KENNEDY Terry	Male	1996-07-04
1533364	ISMAIL Mohamed	Male	1997-01-01
1533371	HASSAN Ibrahim	Male	1997-01-01
1539008	NOVAC Alexandru Mihaita	Male	1997-03-24
1535364	CARRILLO Duilio	Male	1997-03-24
1538028	BOSCO Eugenia	Female	1997-06-12
1537187	FERNANDEZ GAMBOA Pedro Luis	Male	1997-06-12
1539548	TIE Whitney	Female	1999-04-07
1538622	NURMATOV Shakhzod	Male	1999-04-07
1537312	REZAEI Mohammad Nabi	Male	1999-04-10
1537286	MARTINEZ Paulina	Female	1999-04-10
1536479	VAZQUEZ Ana	Female	2000-10-05
1538436	AMINI POZVEH Fatemeh	Female	2000-10-05
1535199	PORTILLO Erick	Male	2000-10-05
1538632	SAFAROLIYEV Sobirjon	Male	2002-05-02
1538007	CROOKS Jordan	Male	2002-05-02
1539607	SIDHU Vijayveer	Male	2002-06-21
1535360	TEJEDA Adirem	Female	2002-06-21
1535505	LINARES Natalia	Female	2003-01-03
1538153	SHKURDAI Anastasiya	Female	2003-01-03

25 rows in set (0.02 sec)

Sample Answer for Question 5

6. What is the completed information of an official?

➔ Understand all the details about the official

```
Select Official.Official_Code, Official.Name, Gender, Belonged_Organization,
Event.Sport_Code,Event.Sport_Name
FROM Official NATURAL JOIN Event_Management NATURAL JOIN Event;
```

Figure 12-Query for Question 6

```
mysql> Select Official.Official_Code, Official.Name, Gender, Belonged_Organization,
-> Event.Sport_Code,Event.Sport_Name
-> FROM Official NATURAL JOIN Event_Management NATURAL JOIN Event;
```

Official_Code	Name	Gender	Belonged_Organization	Sport_Code	Sport_Name
1937701	MESSER KERSTING Thomas	Male	Germany	BK3	3x3 Basketball
1937758	BERTRAND Eric	Male	Switzerland	BK3	3x3 Basketball
1937785	HO Edmond	Male	Hong Kong, China	BK3	3x3 Basketball
1937815	MICHAELIDES Markos-Elias	Male	Switzerland	BK3	3x3 Basketball
1937848	MALISZEWSKI Marek	Male	Poland	BK3	3x3 Basketball
1937858	CHAJIDDINE Najib	Male	France	BK3	3x3 Basketball
1937879	GHIZDAREANU Vlad	Male	Romania	BK3	3x3 Basketball
1937884	JURAS Jasmina	Female	Serbia	BK3	3x3 Basketball
1937903	SHI Qirong	Female	People's Republic of China	BK3	3x3 Basketball
1937919	KIM Gain	Female	Republic of Korea	BK3	3x3 Basketball
1937973	CSABAI-KASKOTO Brigitta	Female	Hungary	BK3	3x3 Basketball
1937982	OKATCH Dorothy	Female	Botswana	BK3	3x3 Basketball
1938013	JACKSON Deanna	Female	United States of America	BK3	3x3 Basketball
1938090	TALISAINEN Alex	Male	Estonia	BK3	3x3 Basketball
1938099	CSENDER Andrada Monika	Female	Romania	BK3	3x3 Basketball
1938104	VESKOVIC Vladimir	Male	Serbia	BK3	3x3 Basketball
1995222	MEARY Aurelia	Female	France	BK3	3x3 Basketball
1995226	BOIRE Elise	Female	France	BK3	3x3 Basketball
1995228	BOUTONNET Gregoire	Male	France	BK3	3x3 Basketball
1995229	PREL Nicolas	Male	France	BK3	3x3 Basketball
1995231	TAEYE Corinne	Female	France	BK3	3x3 Basketball
1995232	FAYS Francois	Male	France	BK3	3x3 Basketball
1995233	FOREST Alexis	Male	France	BK3	3x3 Basketball
1995236	MECHIN Celine	Female	France	BK3	3x3 Basketball
1995237	SEVAUX Arnaud	Male	France	BK3	3x3 Basketball
1995239	CORTINOVIS BEGLOT Chloe	Female	France	BK3	3x3 Basketball

Sample Answer for Question 6

7. Who is the youngest athlete in the Olympic 2024?

➔ Show interesting fact that can be use in news

```
SELECT Athlete_Code, Name, Gender, ROUND((DATEDIFF(CURDATE(), Birth_Date) / 365), 0) AS Age
FROM Athlete
WHERE (DATEDIFF(CURDATE(), Birth_Date) / 365)
<= ALL(SELECT DATEDIFF(CURDATE(), Birth_Date) / 365 FROM Athlete);
```

Figure 13-Query for Question 7

```
mysql> SELECT Athlete_Code, Name, Gender, ROUND((DATEDIFF(CURDATE(), Birth_Date) / 365), 0) AS Age
-> FROM Athlete
-> WHERE (DATEDIFF(CURDATE(), Birth_Date) / 365)
-> <= ALL(SELECT DATEDIFF(CURDATE(), Birth_Date) / 365 FROM Athlete);
+-----+-----+-----+-----+
| Athlete_Code | Name                | Gender | Age |
+-----+-----+-----+-----+
| 1538415      | BEYRANVAND Mohammad | Male   | 16  |
+-----+-----+-----+-----+
```

Sample Answer for Question 7

8. Which events use most days to complete?

➔ The data obtained can be used for scheduling in future Olympic events

```
SELECT Sport_Name, MAX(DATEDIFF(Ended_Date, Started_Date)) AS 'Maximum Days used'
FROM Event NATURAL JOIN Participation
GROUP BY Sport_Name
HAVING MAX(DATEDIFF(Ended_Date, Started_Date)) =
(SELECT MAX(DATEDIFF(Ended_Date, Started_Date)) FROM Event NATURAL JOIN Participation);
```

Figure 14-Query for Question 8

```
mysql> SELECT Sport_Name, MAX(DATEDIFF(Ended_Date, Started_Date)) AS 'Maximum Days used'
-> FROM Event NATURAL JOIN Participation
-> GROUP BY Sport_Name
-> HAVING MAX(DATEDIFF(Ended_Date, Started_Date)) =
-> (SELECT MAX(DATEDIFF(Ended_Date, Started_Date)) FROM Event NATURAL JOIN Participation);
+-----+-----+
| Sport_Name | Maximum Days used |
+-----+-----+
| Football   | 17                |
| Handball   | 17                |
+-----+-----+
```

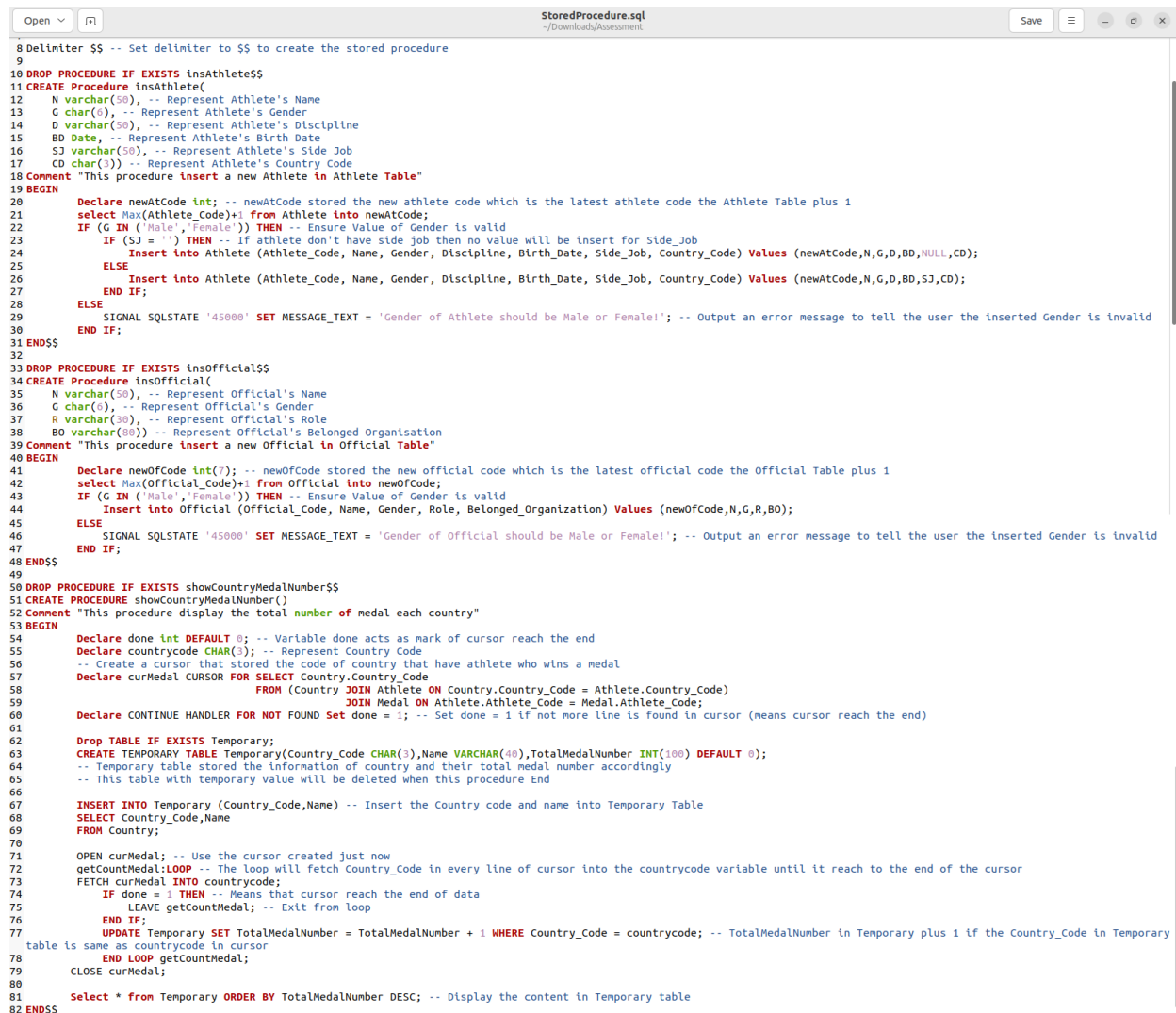
Sample Answer for Question 8

II. Advanced Features

➔ The database has three stored procedure and two triggers

1) Stored Procedure

➔ All stored procedure implementing command are stored in StoredProcedure.sql script and then call by SetUp.sql



```
8 Delimiter $$ -- Set delimiter to $$ to create the stored procedure
9
10 DROP PROCEDURE IF EXISTS insAthlete$$
11 CREATE Procedure insAthlete(
12     N varchar(50), -- Represent Athlete's Name
13     G char(1), -- Represent Athlete's Gender
14     D varchar(50), -- Represent Athlete's Discipline
15     BD Date, -- Represent Athlete's Birth Date
16     SJ varchar(50), -- Represent Athlete's Side Job
17     CD char(3) -- Represent Athlete's Country Code
18 Comment "This procedure insert a new Athlete in Athlete Table"
19 BEGIN
20     Declare newAtCode int; -- newAtCode stored the new athlete code which is the latest athlete code the Athlete Table plus 1
21     select Max(Athlete_Code)+1 from Athlete into newAtCode;
22     IF (G IN ('Male','Female')) THEN -- Ensure Value of Gender is valid
23         IF (SJ = '') THEN -- If athlete don't have side job then no value will be insert for Side_Job
24             Insert into Athlete (Athlete_Code, Name, Gender, Discipline, Birth_Date, Side_Job, Country_Code) Values (newAtCode,N,G,D,BD,NULL,CD);
25         ELSE
26             Insert into Athlete (Athlete_Code, Name, Gender, Discipline, Birth_Date, Side_Job, Country_Code) Values (newAtCode,N,G,D,BD,SJ,CD);
27         END IF;
28     ELSE
29         SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Gender of Athlete should be Male or Female!'; -- Output an error message to tell the user the inserted Gender is invalid
30     END IF;
31 END$$
32
33 DROP PROCEDURE IF EXISTS insOfficial$$
34 CREATE Procedure insOfficial(
35     N varchar(50), -- Represent Official's Name
36     G char(1), -- Represent Official's Gender
37     R varchar(30), -- Represent Official's Role
38     BO varchar(80) -- Represent Official's Belonged Organisation
39 Comment "This procedure insert a new Official in Official Table"
40 BEGIN
41     Declare newOfCode int(7); -- newOfCode stored the new official code which is the latest official code the Official Table plus 1
42     select Max(Official_Code)+1 from Official into newOfCode;
43     IF (G IN ('Male','Female')) THEN -- Ensure Value of Gender is valid
44         Insert into Official (Official_Code, Name, Gender, Role, Belonged_Organization) Values (newOfCode,N,G,R,BO);
45     ELSE
46         SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Gender of Official should be Male or Female!'; -- Output an error message to tell the user the inserted Gender is invalid
47     END IF;
48 END$$
49
50 DROP PROCEDURE IF EXISTS showCountryMedalNumber$$
51 CREATE PROCEDURE showCountryMedalNumber()
52 Comment "This procedure display the total number of medal each country"
53 BEGIN
54     Declare done int DEFAULT 0; -- Variable done acts as mark of cursor reach the end
55     Declare countrycode CHAR(3); -- Represent Country Code
56     -- Create a cursor that stored the code of country that have athlete who wins a medal
57     Declare curMedal CURSOR FOR SELECT Country.Country_Code
58     FROM (Country JOIN Athlete ON Country.Country_Code = Athlete.Country_Code)
59     JOIN Medal ON Athlete.Athlete_Code = Medal.Athlete_Code;
60     Declare CONTINUE HANDLER FOR NOT FOUND Set done = 1; -- Set done = 1 if not more line is found in cursor (means cursor reach the end)
61
62     Drop TABLE IF EXISTS Temporary;
63     CREATE TEMPORARY TABLE Temporary(Country_Code CHAR(3),Name VARCHAR(40),TotalMedalNumber INT(100) DEFAULT 0);
64     -- Temporary table stored the information of country and their total medal number accordingly
65     -- This table with temporary value will be deleted when this procedure End
66
67     INSERT INTO Temporary (Country_Code,Name) -- Insert the Country code and name into Temporary Table
68     SELECT Country_Code,Name
69     FROM Country;
70
71     OPEN curMedal; -- Use the cursor created just now
72     getCountryMedal:LOOP -- The loop will fetch Country_Code in every line of cursor into the countrycode variable until it reach to the end of the cursor
73     FETCH curMedal INTO countrycode;
74     IF done = 1 THEN -- Means that cursor reach the end of data
75         LEAVE getCountryMedal; -- Exit from loop
76     END IF;
77     UPDATE Temporary SET TotalMedalNumber = TotalMedalNumber + 1 WHERE Country_Code = countrycode; -- TotalMedalNumber in Temporary plus 1 if the Country_Code in Temporary
78     table is same as countrycode in cursor
79     END LOOP getCountryMedal;
80     CLOSE curMedal;
81     Select * from Temporary ORDER BY TotalMedalNumber DESC; -- Display the content in Temporary table
82 END$$
```

Figure 15-Content in StoredProcedure.sql

➔ The stored procedure include:

i) insAthlete

➔ Insert new athlete in the Athlete table

```
mysql> call insAthlete('Yee','Male','Archery','2005-04-23','','IND');
Query OK, 1 row affected (0.01 sec)
```

Figure 16-Using insAthlete Stored Procedure

1540143	FISCHER Guillermo	Male	Handball	1996-07-18
1540144	Yee	Male	Archery	2005-04-23
401 rows in set (0.01 sec)				

Advanced Feature Output 1

ii) insOfficial

➔ Insert new official in the Official table

```
mysql> call insOfficial('SecondYee','Male','Judge','HappyLand');
Query OK, 1 row affected (0.01 sec)
```

Figure 17-Using insOfficial Stored Procedure

4968543	CAMPANILE Nicolas	Male	Judge	International Gymnastics Federation
4968544	SecondYee	Male	Judge	HappyLand
546 rows in set (0.00 sec)				

Advanced Feature Output 2

iii) showCountryMedalNumber

➔ Output the countries and their total medals respectively

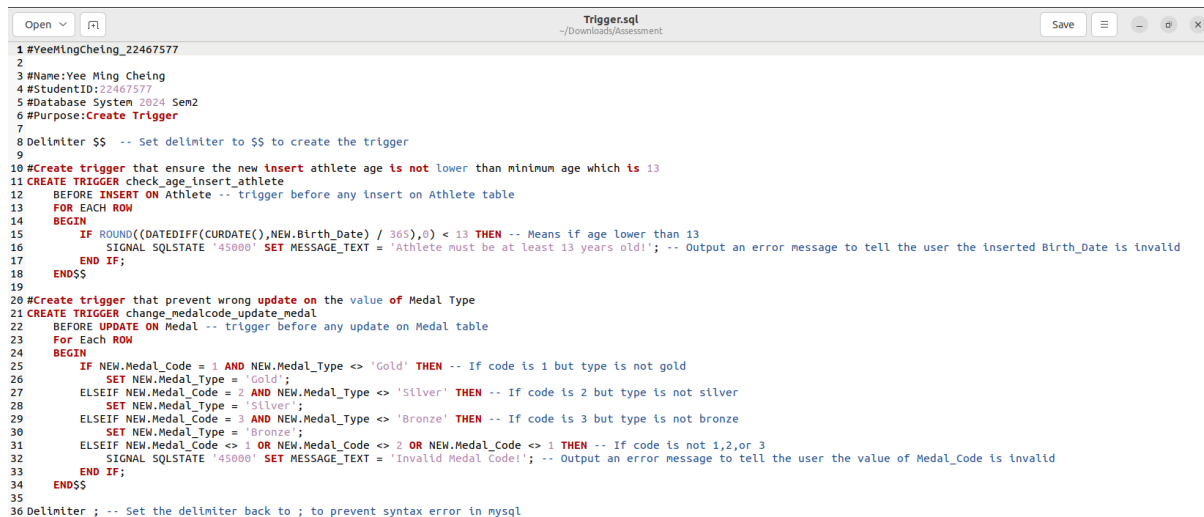
```
mysql> call showCountryMedalNumber();
```

Country_Code	Name	TotalMedalNumber
MEX	Mexico	3
ETH	Ethiopia	2
LCA	Saint Lucia	2
IRL	Ireland	1
JAM	Jamaica	1
PUR	Puerto Rico	1
AFG	Afghanistan	0
AHO	Netherlands Antilles	0
AIN	AIN	0

Advanced Feature Output 3

2) Triggers

➔ All triggers implementing command are stored in Trigger.sql script and then call by SettingUp.sql



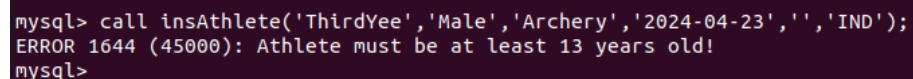
```
1 #YeeHingCheing_22467577
2
3 #Name:Yee Ming Cheing
4 #StudentID:22467577
5 #Database System 2024 Sem2
6 #Purpose:Create Trigger
7
8 Delimiter $$ -- Set delimiter to $$ to create the trigger
9
10 #Create trigger that ensure the new insert athlete age is not lower than minimum age which is 13
11 CREATE TRIGGER check_age_insert_athlete
12 BEFORE INSERT ON Athlete -- trigger before any insert on Athlete table
13 FOR EACH ROW
14 BEGIN
15     IF ROUND((DATEDIFF(CURDATE(),NEW.Birth_Date) / 365),0) < 13 THEN -- Means if age lower than 13
16         SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Athlete must be at least 13 years old!'; -- Output an error message to tell the user the inserted Birth_Date is invalid
17     END IF;
18 END$$
19
20 #Create trigger that prevent wrong update on the value of Medal Type
21 CREATE TRIGGER change_medalcode_update_medal
22 BEFORE UPDATE ON Medal -- trigger before any update on Medal table
23 FOR EACH ROW
24 BEGIN
25     IF NEW.Medal_Code = 1 AND NEW.Medal_Type <> 'Gold' THEN -- If code is 1 but type is not gold
26         SET NEW.Medal_Type = 'Gold';
27     ELSEIF NEW.Medal_Code = 2 AND NEW.Medal_Type <> 'Silver' THEN -- If code is 2 but type is not silver
28         SET NEW.Medal_Type = 'Silver';
29     ELSEIF NEW.Medal_Code = 3 AND NEW.Medal_Type <> 'Bronze' THEN -- If code is 3 but type is not bronze
30         SET NEW.Medal_Type = 'Bronze';
31     ELSEIF NEW.Medal_Code <> 1 OR NEW.Medal_Code <> 2 OR NEW.Medal_Code <> 3 THEN -- If code is not 1,2,or 3
32         SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Invalid Medal Code!'; -- Output an error message to tell the user the value of Medal_Code is invalid
33     END IF;
34 END$$
35
36 Delimiter ; -- Set the delimiter back to ; to prevent syntax error in mysql
```

Figure 18-Content in Trigger.sql

➔ The stored procedure include:

i) check_age_insert_athlete

➔ Check for the user inserted birth date's validity



```
mysql> call insAthlete('ThirdYee','Male','Archery','2024-04-23','','IND');
ERROR 1644 (45000): Athlete must be at least 13 years old!
mysql>
```

Figure 19-Evidence of Trigger 1 implementation

ii) change_medalcode_update_medal

➔ Ensure user updated medal's type follow the medal code



```
mysql> update Medal Set Medal_Code =3 where Athlete_Code=1539986;
Query OK, 1 row affected (0.02 sec)
Rows matched: 1 Changed: 1 Warnings: 0
```

1537325	1	Gold	2024-08-03
1537325	2	Silver	2024-08-06
1539986	3	Bronze	2024-08-03

10 rows in set (0.00 sec)

Figure 20-Evidence of Trigger 2 implementation

III. Python implementation

The implementation of python program is related to three files:

i) Background.sql

➔ A SQL script call by SettingUp.sql that set up the user's name and the password in MySQL to use the Python program

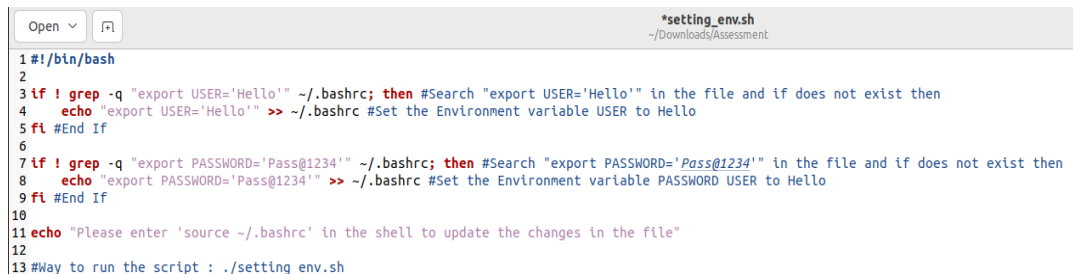


```
1 #YeeMingCheing_22467577
2
3 #Name:Yee Ming Cheing
4 #StudentID:22467577
5 #Database System 2024 Sem2
6 #Purpose:Set up the User and the password
7
8 DROP USER 'Hello'@'localhost'; -- Drop the Hello User if exist
9
10 CREATE USER 'Hello'@'localhost' IDENTIFIED BY 'Pass@1234'; -- Create the Hello User with Password
11
12 GRANT ALL PRIVILEGES ON Olympic_22467577.* TO 'Hello'@'localhost'; -- Grant permissions to the Hello User for the database
13
14 FLUSH PRIVILEGES; -- Update the Changes
```

Figure 21-Content in Background.sql

ii) setting_env.sh

➔ A bash scrip manually calls by user to set the environment variable USER and PASSWORD to create a secure connection between the Python program to database



```
1 #!/bin/bash
2
3 if ! grep -q "export USER='Hello'" ~/.bashrc; then #Search "export USER='Hello'" in the file and if does not exist then
4     echo "export USER='Hello'" >> ~/.bashrc #Set the Environment variable USER to Hello
5 fi #End If
6
7 if ! grep -q "export PASSWORD='Pass@1234'" ~/.bashrc; then #Search "export PASSWORD='Pass@1234'" in the file and if does not exist then
8     echo "export PASSWORD='Pass@1234'" >> ~/.bashrc #Set the Environment variable PASSWORD USER to Hello
9 fi #End If
10
11 echo "Please enter 'source ~/.bashrc' in the shell to update the changes in the file"
12
13 #Way to run the script : ./setting_env.sh
```

Figure 22-Content in setting_env.sh

iii) ConnectOlympic.py

➔ The Python program that connects to the Olympic_22467577 database which provide some useful functions such as delete or update athlete's information

```
Open  ConnectOlympic.py
~/Downloads/Assessment

1 #YeeMingCheing_22467577
2
3 #Name:Yee Ming Cheing
4 #StudentID:22467577
5 #Database System 2024 Sem2
6 #Purpose:Python Connect to Database Olympic_22467577
7
8 import mysql.connector #Import mysql connector
9 import os #Import Operating System library (access environment variables)
10
11 #Get the password and user name from environment variables
12 DPASSWORD = os.getenv('PASSWORD')
13 DUSER = os.getenv('USER')
14
15 #Connect to Olympic_22467577 Database
16 db = mysql.connector.connect(
17     host = "localhost", #hostname
18     user = DUSER, #username
19     passwd = DPASSWORD, #userpassword
20     database = "Olympic_22467577") #databasename
21
22 #Create a cursor object to execute the queries
23 mycursor=db.cursor()
24
25 #Prepare the query
26 ShowAthlete = ("SELECT * FROM Athlete") #Display all the athlete
27 ShowWinner = ("SELECT * FROM Winner") #Display all the winner
28 InsertAthlete = ("CALL insAthlete(%s, %s, %s, %s, %s, %s)") #Insert an athlete using stored procedure
29 DeleteAthlete = ("DELETE FROM Athlete WHERE Athlete_Code = %s") #Delete specific athlete
30 UpdateAthlete = ("UPDATE Athlete SET Discipline =%s WHERE Athlete_Code = %s") #Update discipline of a specific athlete
31 #Display the code and the name of the athlete that has a side job and was born between 1995 and 2005
32 SampleQueryOne = ("SELECT Athlete_Code,Name,Side_Job FROM Athlete WHERE Side_Job IS NOT NULL AND Birth_Date BETWEEN '1995-01-01' AND '2005-12-31'")
33 #Display the code, the name, and the role of official come from a Sport Federation
34 SampleQueryTwo = ("SELECT Official_Code,Name,Role FROM Official WHERE Belonged_Organization LIKE '%Federation%'")
35
```

Figure 23-Section of content in ConnectOlympic.py

```
mysql> exit
Bye
yee@yee-virtual-machine:~/Downloads/Assessment$ python3 ConnectOlympic.py

Welcome
Please select one of the following option
1)Show Athlete Table
2)Show Winner Table
3)Insert a athlete into Athlete Table
4)Delete a athlete into Athlete Table
5)Update a athlete's disipline into Athlete Table
6)QueryOne(Display information of the athlete that has a side job and was born between 1995 and 2005)
7)QueryTwo(Display information of official from a Sport Federation)
0)Exit
Enter your choice: 6
(1533136, 'BASS BITTAYE Gina Mariam', 'police officer (sub-inspector)')
(1533176, 'CAMARA Ebrahima', 'prison officer')
(1533235, 'ARELLANO Fernanda', 'student')
(1533237, 'TOSCANO Pamela', 'sport soldier')
(1533364, 'ISMAIL Mohamed', 'sport soldier')
(1533371, 'HASSAN Ibrahim', 'soldier with the Djiboutian Republican Guard')
(1533372, 'WAISS Abdi', 'sport soldier')
(1533374, 'HASSAN NOUR Samiyah', 'sport soldier')
(1535191, 'GONZALEZ Alegna', 'serves in the navy')
(1535226, 'AGUILAR CHAVEZ PEON Mariana', 'student')
(1535310, 'BLANCO Luisa', 'student')
(1535343, 'WILLARS VALDEZ Randal', 'Armed forces ')
(1535346, 'OROZCO LOZA Alejandra', 'Armed forces ')
(1535348, 'AGUNDEZ GARCIA Gabriela', 'Armed forces ')
(1535364, 'CARRILLO Duilio', 'Armed Forces ')
```

Figure 24-Python Program successfully connect to the database

Discussion

A. Reflection

This is my first time creating a database system, and it took me 2 weeks to complete. In the beginning, I started with online research on the Olympics 2024 and got some useful information, including the data source. After that, I spent two days thinking of the ER model's draft and drawing it out. The draft helps me a lot in choosing the primary key and the data type of the attribute of each entity during the designing of the real database structure.

After creating the ER modal, I created the relational schema and implemented the database using MySQL. I found the database implementation quite straightforward, as I have done many lab exercises. In the following days, I created several queries, stored procedures, triggers, and a Python program. This process was interesting, as it clarified some of the concepts that I was struggling with during the lab session. Finally, I document the user guide and report, eventually completing my final assessment.

This assessment provides me with some valuable insights, such as the importance of having a draft before starting a project and the necessity of having a systematic plan to do the assessment.

B. Issues faced and solutions

Several challenges arose during the design and implementation of the database. Initially, I created 15 entities for my database and tried to implement them into the ER model. However, it was difficult for me to proceed with the next step; handling so many entities is more complex than I expected. Thus, I solve this problem by filtering the unnecessary entities, such as the schedule or event's category, and eventually reducing the total entities from 15 to 6, which allows me to effectively manage the ER Model and also the relational schema.

Another challenge was writing a Python program to connect to my database, as I had no experience in using Python. I discovered the syntax of Python is quite different from the programming languages I have learned, such as C++ or Java. To solve this, I visit some social media websites and watch tutorials on basic Python programming. I also researched ways to prevent hardcoding the user's name and password in the programming script. I spent 4 days repeatedly testing my Python program, and finally, it performed well.

Each of these challenges provided an opportunity for growth and a reminder for me to prepare well before doing the assessment in the future.

C. Future Directions

Due to the time limits, I was unable to cover or add some other features that I originally planned to my database system. During the implementation of the Python program, I was thinking about implementing a website that connects to my database. This website will provide a user-friendly interface for the users, especially those unfamiliar with Linux OS, to easily manage the Olympic data and use them in the research.

Besides, the second feature I want to add is the feedback system. The database will allow users to report bugs and also give suggestions. This feedback will be sent to my email every week and become a valuable reference for me to update the database system. With the help of the users, I believe that the database system will become more and more efficient and eventually become a well-known resource in sports research.

The last feature I hope to add is the import function. The user can import the excel file (.csv) to effectively add a large amount of data to the table. This feature not only increases the user's convenience but also improves its reusability.

References

- Filip, P. (2024). *Paris 2024 Olympic Summer Games* [Data set]. KAGGLE.
<https://www.kaggle.com/datasets/piterfm/paris-2024-olympic-summer-games>
- Python. (n.d.). *8. Errors and Exceptions*.
<https://docs.python.org/3/tutorial/errors.html>
- MySQL. (n.d.). *15.6.7.5 SIGNAL Statement*.
<https://dev.mysql.com/doc/refman/8.4/en/signal.html>
- Hira, Z. (2022, October 26). *In programming, you use variables to store information like strings and numbers temporarily. Variables can be used repeatedly throughout the* [Article]. freeCodeCamp.
<https://www.freecodecamp.org/news/how-to-set-an-environment-variable-in-linux/>
- freeCodeCamp.org. (2023, April 11). *Bash Scripting Tutorial for Beginners* [Video]. YouTube. <https://www.youtube.com/watch?v=tK9Oc6AEnR4>